

Hands-On-Learning: Human-in-the-Loop Interface Design Challenge

Learning Objectives

By the end of this activity, learners will be able to:

- Design a simple chat UI that clearly distinguishes user and model messages.
- Integrate human-in-the-loop controls (rating, flagging, clarification) into each model response.
- Capture feedback data in a structured way and send it to a backend service.
- Display feedback signals visually (e.g., counts, simple stats) to demonstrate a working feedback loop.
- Explain how specific UX choices influence user trust and perceived model quality.



Description

In this hands-on practice, you will act as a UX-minded frontend engineer designing a feedback-aware interface for an LLM app. You'll build a minimal React chat UI using the Vercel AI SDK, add feedback controls to each response, send those signals to a FastAPI/Node backend, and visualize basic feedback metrics. The goal is not just to collect feedback, but to show how interface design makes the AI feel more transparent and trustworthy.

Structure of the Activity

Title: Hands-on Practice: Human-in-the-Loop Interface Design Challenge

Dataset information / Resources Required:

- No traditional dataset required.
- Simple mock or real LLM backend endpoint (e.g., /api/chat) from earlier module's lab or a provided starter.
- Optional: small in-memory feedback store on the backend to return basic stats.

Tools Used:

- **Frontend:** React
- **LLM UI Helpers:** Vercel AI SDK
- **Backend:** FastAPI (Python) or Node.js (Express/Nest)
- **Optional:** Any simple charting or plain JSX for basic visual stats

Setup Requirements:

- Node.js environment with React app scaffolded (e.g., create-next-app or create-react-app).
- Vercel AI SDK installed and configured for your chat route.
- FastAPI or Node.js backend running locally with at least:
 - A chat endpoint (/api/chat) that returns model responses.
 - A feedback endpoint (/api/feedback) to accept feedback payloads.

Scenario

You're designing the interface for a customer-facing LLM app that occasionally produces uncertain or slightly off answers. Product stakeholders don't just want "better prompts"—they want visible transparency and a way for users to speak up when the AI gets things wrong.

They've asked for three things:

- Simple, intuitive ways for users to rate or flag responses.
- A way for users to request clarification or explain why something felt wrong.

- A small visual indicator or metric that proves the system is actually collecting feedback, not just pretending to.

Your task is to build a minimal chat UI that:

- Shows conversation clearly.
- Adds feedback controls to each model response.
- Sends feedback data to the backend and displays simple feedback stats.

Objective

Create an interactive UI prototype that demonstrates a working feedback loop between end users and the LLM system.



Instructions for Participants

Getting Started

1

Open your React project (Next.js or CRA) where the Vercel AI SDK is available.

2

Ensure your backend is running with:

→ A chat route (e.g., /api/chat or /api/llm) returning model responses.

→ A feedback route (e.g., /api/feedback) that accepts POST requests with JSON payload.

3

Create or open a page/component for the chat interface (e.g., ChatPage.jsx or app/page.tsx).

Activity 1: Build the Chat Interface Skeleton

Objective

Create a minimal but clear chat interface that separates user and assistant messages and supports streaming responses.



Input/Data:

- Existing or simple LLM backend endpoint.
- Vercel AI SDK hooks (e.g., useChat).

Steps to Complete

Step 1: Wire Up Basic Chat Using Vercel AI SDK

- Use useChat (or equivalent) to:
 - Handle user input.
 - Send messages to your backend.
 - Receive and render streamed responses.

Step 2: Render Message List

- Display messages in a simple vertical list.
- Clearly distinguish:
 - User messages (e.g., right aligned, different background).
 - Assistant messages (e.g., left aligned, neutral background).
- Include timestamps or small metadata if desired.

Step 3: Add Input Box and Send Button

- Provide a text input at the bottom.
- Allow pressing “Enter” or clicking a button to send.
- Clear the input after sending, and show the streaming response in the list.

Expected Outcome:

- A functioning chat UI where a user can send a message and see model responses streamed or displayed.

Tips:

- Keep styling minimal; clarity > polish.
- Ensure each assistant message has a stable identifier (e.g., index or message id) for feedback mapping.

Activity 2: Add Feedback Controls and Visualize Signals

Objective

Attach feedback controls to each assistant response, send feedback to the backend, and display aggregate signals in the UI.



Input/Data:

- Assistant messages rendered in Activity 1.
- Feedback endpoint on the backend (stub is fine initially), e.g., POST /api/feedback.

Steps to Complete

Step 1: Embed Feedback Controls in Each Assistant Message

For each assistant message:

- Add **thumbs up / thumbs down buttons**, or equivalent icons.
- Add a small “**Flag**” or “Something’s wrong” button.
- Optionally add a link/button to “**Explain why**” that opens a small text field.

Design considerations:

- Keep controls compact and close to the message they refer to.
- Use simple labels or icons with tooltips for clarity.

Step 2: Send Feedback Data to Backend

→ On click of thumbs up/down or flag:

Construct a structured JSON payload, e.g.:

```
{  
  "messageId": "<id or index>",  
  "rating": "up" | "down" | null,  
  "flagged": true | false,  
  "comment": "optional explanation",  
  "timestamp": "<ISO date>",  
  "sessionId": "<random session id>"  
}
```

→ Send this payload via fetch or Axios to /api/feedback.

Handle basic success/failure states (e.g., small “Feedback saved” or “Try again” message).

Step 3: Track and Display Simple Feedback Metrics

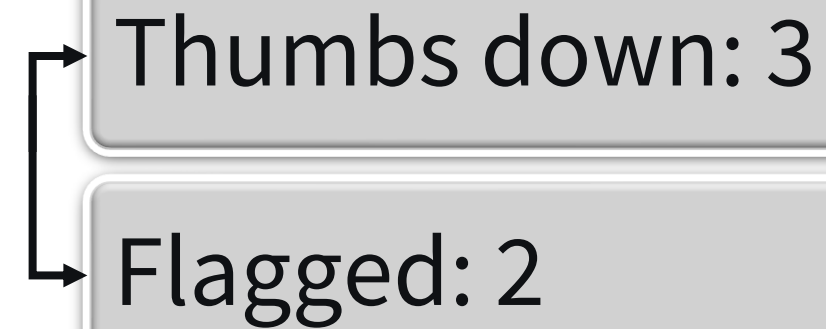
On the backend, aggregate feedback in a simple in-memory store (e.g., counts of thumbs up/down per session or overall).

Add a frontend call to fetch basic stats (e.g., GET /api/feedback/stats) or return updated stats in the feedback response.

Display a small “Feedback Summary” panel in the UI, for example:

Total responses: 10

Thumbs up: 7



Thumbs down: 3

Flagged: 2

Implementation can be very simple—text is fine; no need for fancy charts unless you want them.

Expected Outcome

Each assistant message has interactive feedback controls.

Feedback is sent and stored on the backend.

The UI displays at least one visible feedback metric (counts, percentages, etc.), proving that the loop works.

Tips:

- If time is short, focus on thumbs up/down plus a total count metric.
- Use a randomly generated session id to separate different runs if you'd like.
- Remember: the goal is to demonstrate a feedback loop, not build a full analytics dashboard.

Compile Your Findings

1 Functional UI Prototype

Deliverable: Frontend + backend code showing real feedback capture.

Include:

→ React component/page for the chat UI with feedback controls wired up.

→ Backend routes for `/api/chat` and `/api/feedback` (and optional `/api/feedback/stats`).

→ Any simple in-memory data structure used to store feedback.

2

Screenshot / Short Screen Recording

→ **Deliverable:** Visual proof of feedback interaction.

→ **Include:**

→ A screenshot of the UI showing at least one assistant message, its feedback buttons, and the summary metrics.

→ OR a short video (screen recording) of:

→ Sending a message, receiving an answer, clicking feedback, and seeing metrics update.

3

Reflection

→ **Deliverable:** Written reflection on UX choices.

→ **Include:**

→ Why you placed feedback controls where you did.

→ How your design makes it easy for users to rate or flag answers| without friction.

→ How the visible metrics or feedback summary help users feel that their input matters.

→ What you would add next (e.g., showing confidence indicators, exposing reasoning, or surfacing a “Why this answer?” section) to further boost trust.

Summary

This activity gives you practical experience turning a static chat UI into a **feedback engine**. Instead of just displaying answers, your interface now:

- Lets users react (thumbs, flags, clarifications) in a low-friction way.
- Sends those signals to a backend, proving that the system listens—not just talks.
- Surfaces basic metrics so both users and product teams can see how the assistant is performing.

Key takeaways

- UX defines how “smart” and “trustworthy” the model feels, even if the underlying model didn’t change.
- Human-in-the-loop feedback is a design pattern, not just a logging feature.
- Even small, visible feedback loops (icons + simple counts) can dramatically increase user trust and engagement.

Resources

Primary Materials:

- Course example code from Module 3: Vercel AI SDK chat setup and basic FastAPI/Node backend.

Reference Documentation:

- Vercel AI SDK docs (for useChat and streaming patterns).
- React documentation for component state and event handling.
- FastAPI or Express/Nest documentation for building feedback endpoints.

Additional Resources (Optional):

- Google PAIR Guidebook: patterns for human-centered AI design.
- Articles/examples of feedback-driven AI interfaces (e.g., chatbots with rating systems).

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.